

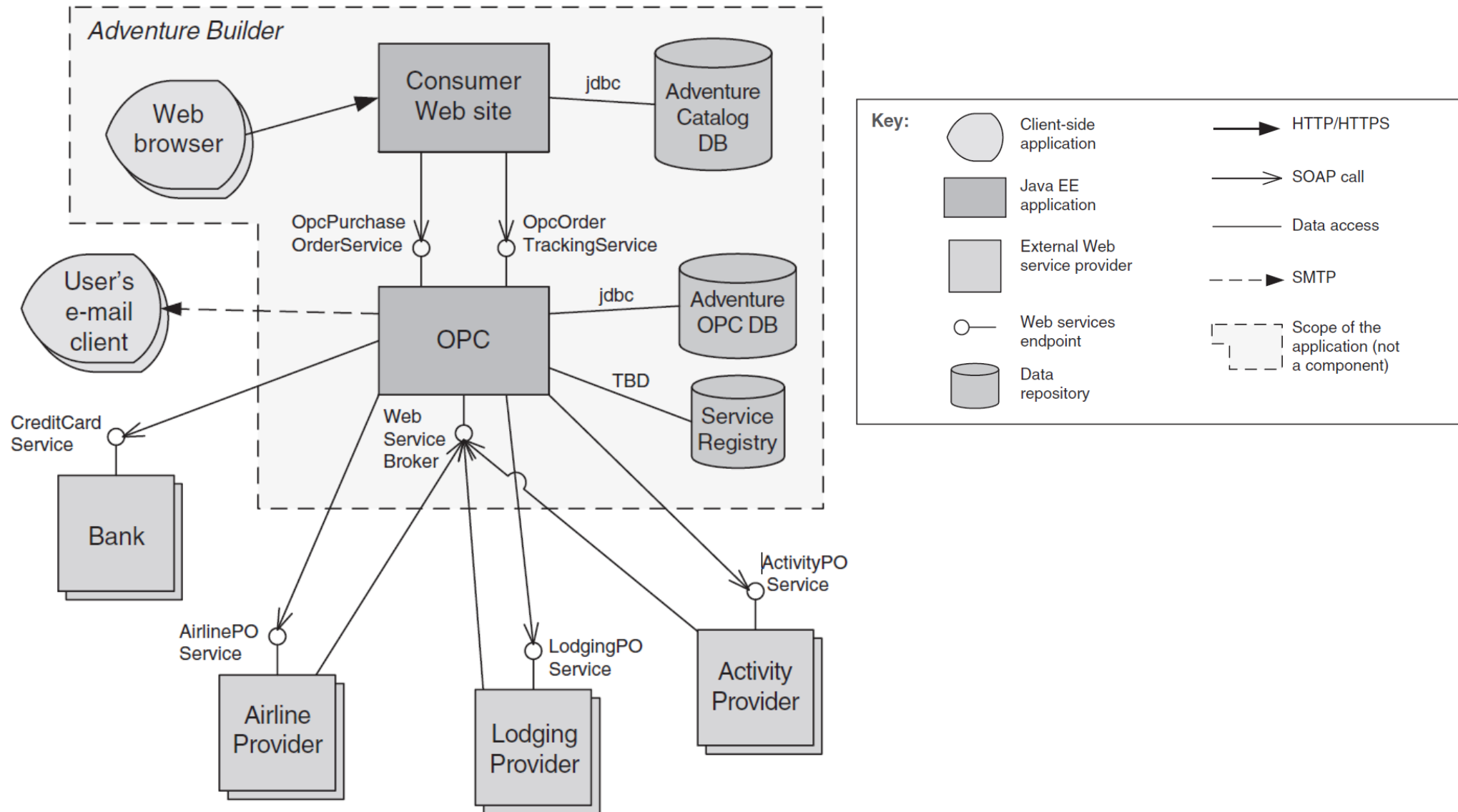
Component-and-connect patterns:

- Broker
- Model-view Controller
- Pipe and Filter
- Client-Server
- Peer-to-Peer
- **Service-Oriented-Architecture**
- Publish-Subscribe
- Shared-Data

Service Oriented Architecture Pattern

- **Context:** A number of **services** are **offered** (and described) **by service providers** and **consumed by service consumers**. Service consumers **need** to be able to **understand** and use these **services without** any **detailed knowledge** of their **implementation**.
- **Problem:** **How** can we **support interoperability** of **distributed components** running on **different platforms** and written in different **implementation languages**, provided by different **organizations**, and distributed across the Internet?
- **Solution:** The **service-oriented architecture (SOA)** pattern describes a collection of **distributed components** that **provide** and/or **consume services**.

Service Oriented Architecture Example



Service Oriented Architecture Solution - 1

- Overview: Computation is achieved by a set of cooperating components that provide and/or consume services over a network.
- **Elements:**
 - Components:
 - *Service providers*, which provide one or more services through published interfaces.
 - *Service consumers*, which invoke services directly or through an intermediary.
 - *Service providers* may also be service consumers.
- SOA application may use specialized components that act as intermediaries and provide infrastructure services
 - *ESB (enterprise service bus)*
 - *Registry of services*
 - *Orchestration server*

Service Oriented Architecture Solution - 2

- SOA application **may use specialized components** that act as intermediaries and provide infrastructure services
 - *ESB* (*enterprise service bus*), which is an **intermediary element** that can:
 - **route and transform messages between service providers and consumers.**
 - **convert messages from one protocol or technology to another**
 - **perform various data transformations** (format, content, splitting, merging)
 - **perform security checks**
 - **ESB promotes interoperability, security, and modifiability**
 - **Communicating through an ESB adds overhead thereby lowering performance**, and introduces an additional **point of failure**
 - **When an ESB is not in place**, service providers and consumers communicate with each other in a **point-to-point** fashion.

Service Oriented Architecture Solution - 3

- SOA application **may use specialized components** that act as intermediaries and provide infrastructure services
 - *Registry of services*, which **may be used** by **providers** to **register** their **services** and by **consumers** to **discover services** at **runtime**.
 - Registry of services:
 - improves the independence of service providers
 - increases system modifiability by hiding the location and identity of the service provider

Service Oriented Architecture Solution - 4

- SOA application **may use specialized components** that act as intermediaries and provide infrastructure services
 - *Orchestration server*, which **coordinates** the **interactions** between **service consumers** and **providers** based on **languages** for business processes and workflows.
 - It executes scripts upon the occurrence of a specific event (e.g., a purchase order request arrived).

Service Oriented Architecture Solution - 5

- **Connectors:**

- *SOAP connector*, which uses the SOAP protocol for synchronous communication between web services, typically over HTTP.
 - *Representational State Transfer (REST) connector*, which relies on the basic request/reply operations of the HTTP protocol.
 - *Asynchronous messaging connector*, which uses a messaging system to offer point-to-point or publish-subscribe asynchronous message exchanges.
- In practice, SOA environments may involve a mix of the three connectors

Service Oriented Architecture Solution - 6

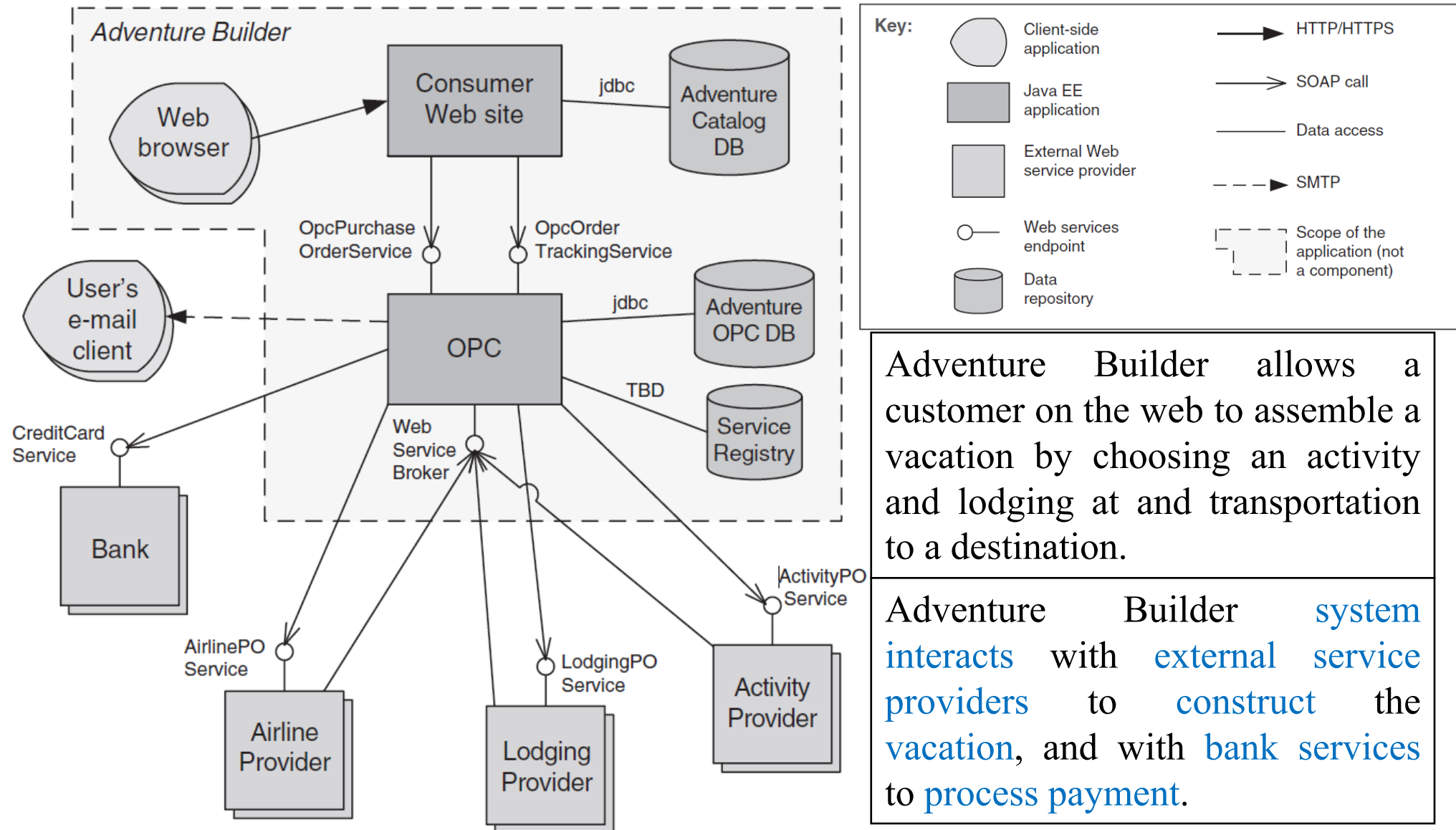
- **Relations:** Attachment of the different kinds of components available to the respective connectors
- **Constraints:** Service consumers are connected to service providers, but intermediary components (e.g., ESB, registry, orchestration server) may be used.
- **Weaknesses:**
 - SOA-based systems are typically complex to build.
 - You don't control the evolution of independent services.
 - There is a performance overhead associated with the middleware, and services may be performance bottlenecks, and typically do not provide performance guarantees.

Service Oriented Architecture Solution - 7

- **Advantages and Benefits:**

- **interoperability.** Because service providers and service consumers may run on different platforms, service-oriented architectures often integrate a variety of systems, including legacy systems.
- SOA also **offers** the necessary **elements** to **interact** with **external services** available over the Internet
- **Special SOA components** such as the **registry** or the **enterprise service bus ESB** also **allow dynamic reconfiguration**, which is **useful when** there's a **need** to **replace** or **add versions** of **components with no system interruption**

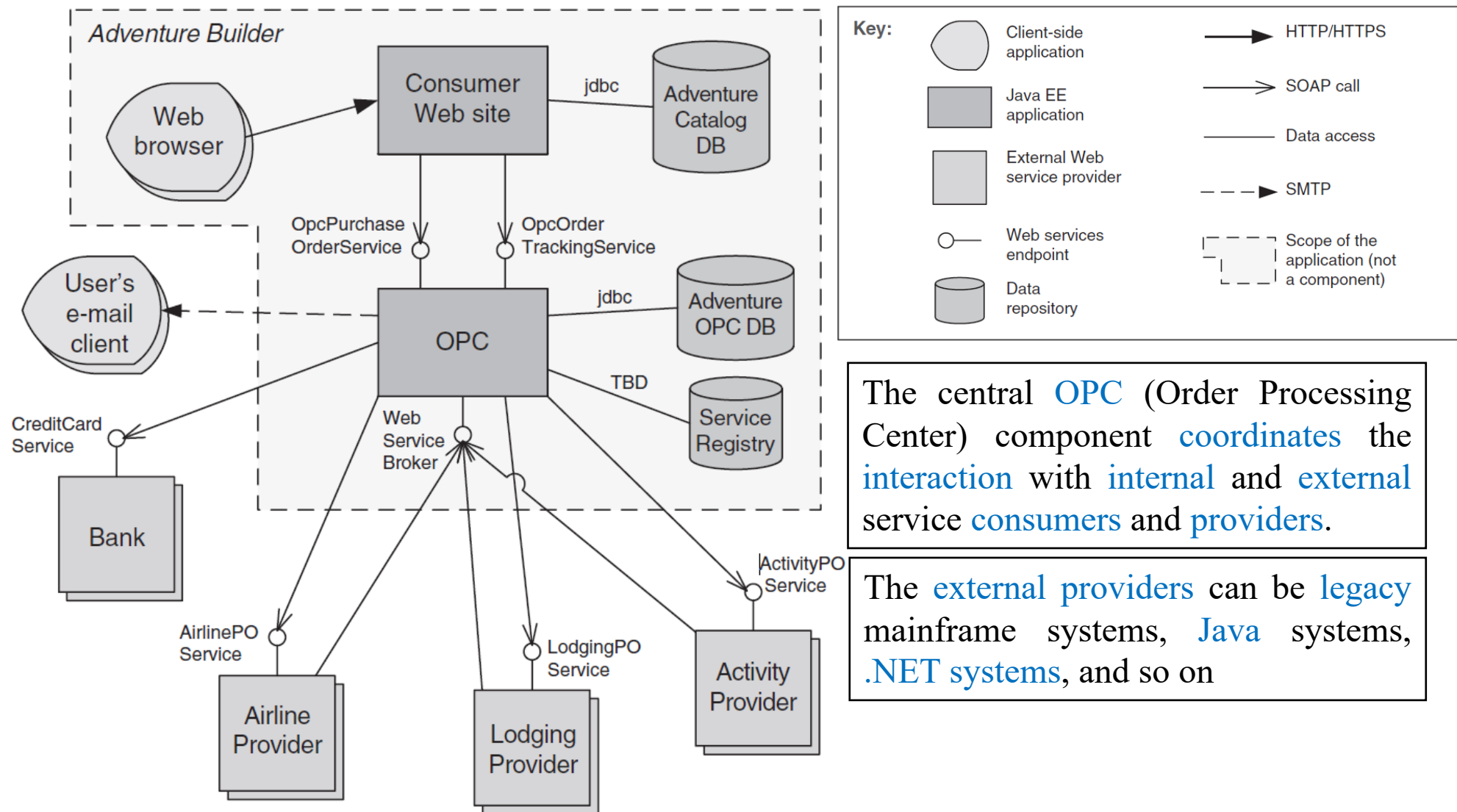
Service Oriented Architecture Example: the Adventure Builder system



Adventure Builder allows a customer on the web to assemble a vacation by choosing an activity and lodging at and transportation to a destination.

Adventure Builder system interacts with external service providers to construct the vacation, and with bank services to process payment.

Service Oriented Architecture Example: the Adventure Builder system



The central **OPC** (Order Processing Center) component **coordinates** the **interaction** with **internal** and **external** service **consumers** and **providers**.

The **external providers** can be **legacy** mainframe systems, **Java** systems, **.NET** systems, and so on